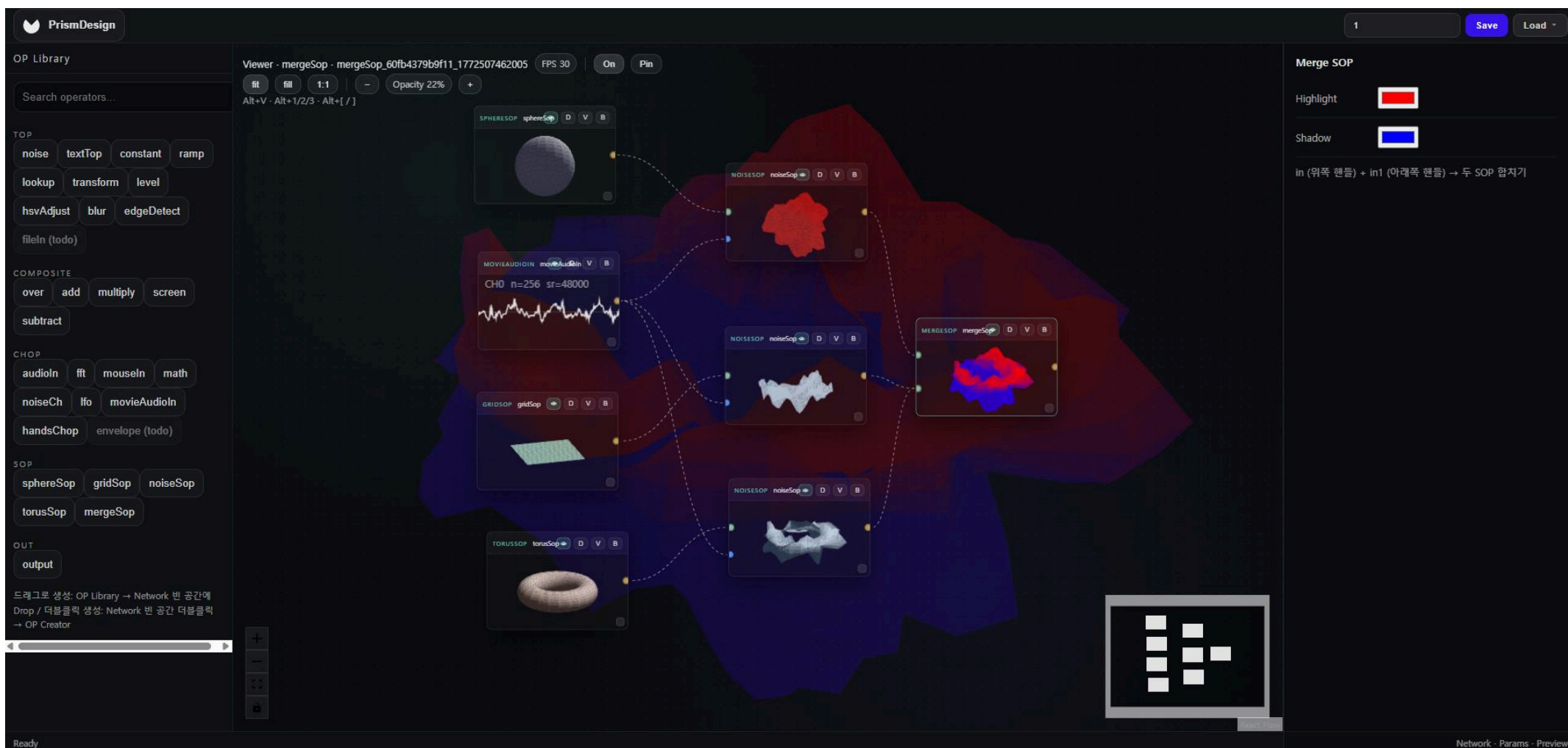




서비스

TOP·CHOP·SOP 오퍼레이터를 노드 그래프로 연결하고, 각 단계의 결과와 최종 비주얼을 실시간으로 확인할 수 있는 웹 기반 비주얼 프로그래밍 도구입니다.

[PrismDesign 바로가기](#)

[디자인 링크 \(피그마\)](#)

[GitHub](#)

작업 기간

2026. 02 - 2026. 03 (1개월)

기술 스택

 React  TypeScript  ReactFlow  Figma

 Vite  MediaPipe Tasks Vision

 Node.js  express Express

역할

Product Design

프로젝트 기획

프로젝트 매니징

Frontend

프로토타입 제작 (figma)

UI/UX 디자인 [반응형_Mobile, PC) (figma)

PM

프론트엔드 개발 일정 관리 및 업무 배정

컴포넌트, 라우트 구조, 레이아웃 설계

공용 컴포넌트 구현 (Form, Modal)

캘린더 구현 (캘린더 공유, 과제 관리)

알림센터 구현 (과제, 채팅)

구성원

PM & FE (박성진)

BE & FE (신동화)

프로젝트 회고

- 데스크톱 비주얼 프로그래밍 방식을 브라우저 환경에 맞게 재구성
- 노드 연결을 단순 UI가 아닌 실제 실행 가능한 데이터 흐름으로 설계
- 팀장으로 프로젝트를 이끌었음
 - 첫 프로젝트 팀원에 눈높이에 맞게 진행
- 실시간 입력을 CHOP 채널로 추상화해 다양한 오퍼레이터에서 재사용
- PM·디자인·프론트엔드 역할을 병행하며 제품 단위로 구현

1. 시스템 아키텍처

React Client



Studio State

Nodes / Edges

Operator Params

Selection

Viewer State

Runtime / Evaluator

Input Map

Registry 조회

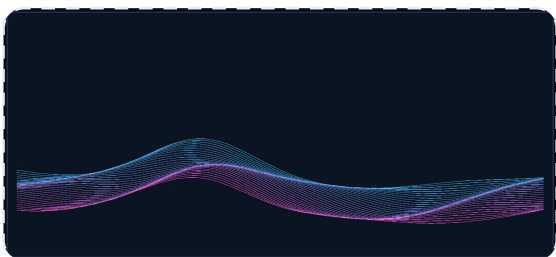
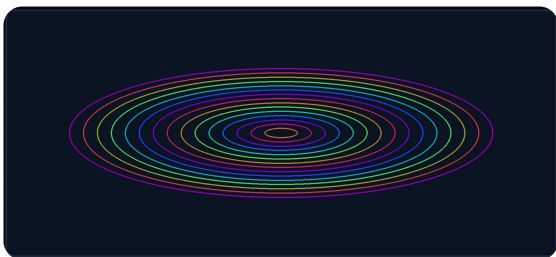
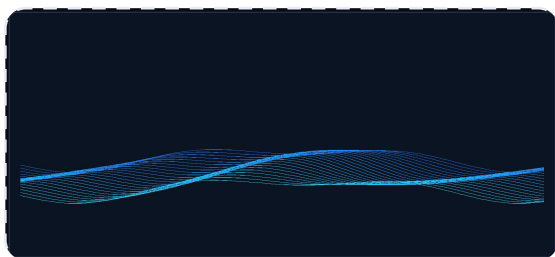
TOP 평가

CHOP 평가

SOP 평가

Frame Cache

Canvas 2D Preview / Final Viewer



Backend (Express)

JWT 인증

Bearer Token 검증
사용자 식별

Graph CRUD API

생성 · 조회 · 저장
이름 변경 · 삭제

User Graph Directory

사용자별 폴더 구조

JSON Graph Storage

/graphs

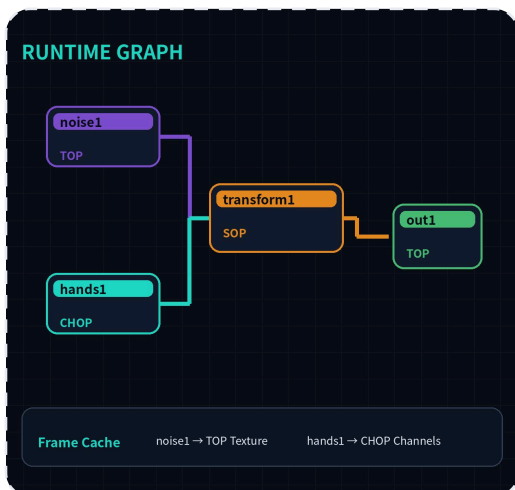
```
{userId}
├── graph_001.json
└── graph_002.json
```

JSON GRAPH STORAGE

```
{
  "graphs": [
    "graph_001.json",
    "graph_002.json"
  ],
  "nodes": [
    "noise1",
    "transform1",
    "SOP",
    "circle1",
    "out1"
  ],
  "edges": [
    "noise1 -> transform1",
    "hands1 -> transform1",
    "transform1 -> SOP",
    "SOP -> circle1",
    "circle1 -> out1"
  ],
  "params": {
    "noise1": {
      "amplitude": 0.72,
      "frequency": 0.46,
      "speed": 0.63,
      "seed": 0.34,
      "feedback": 0.81
    }
  },
  "metadata": {
    "name": "Particle Wave",
    "updatedAt": "2025-08-02"
  }
}
```

그래프 평가 및 실시간 출력 흐름

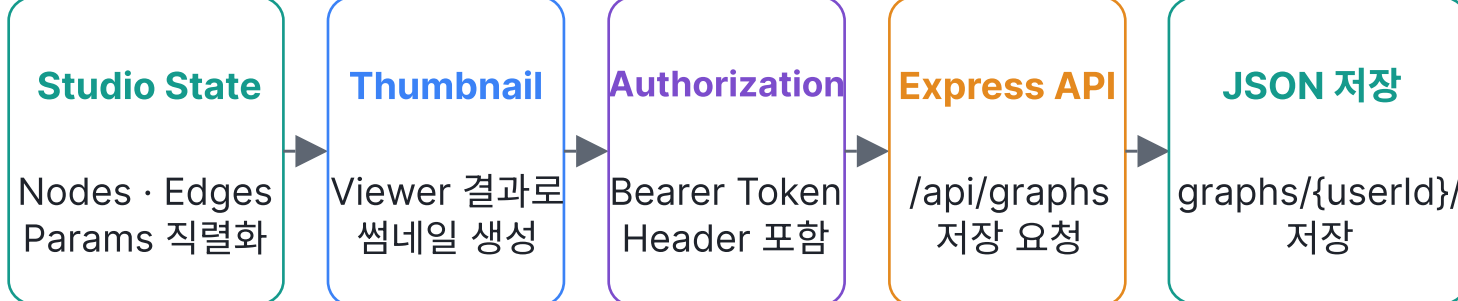
- 1 노드 생성 및 연결
- 2 Edge 기반 Input Map 생성
- 3 연결된 입력 노드 탐색
- 4 Operator Registry 조회
- 5 TOP · CHOP · SOP 평가
- 6 프레임 단위 결과 캐싱
- 7 Preview 및 Viewer 출력



Frame Cache

noise1 → TOP Texture
hands1 → CHOP Channels
out1 → TOP Texture

사용자별 프로젝트 저장 흐름



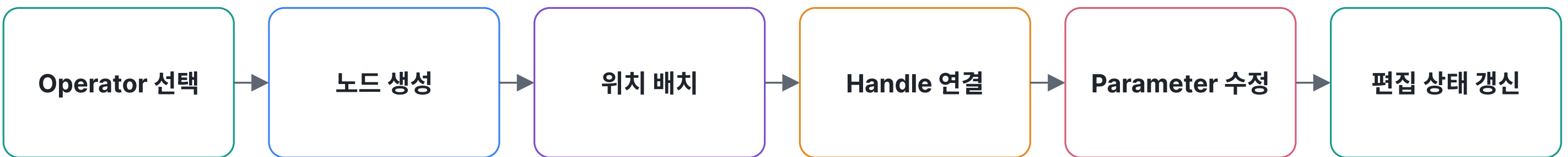
API 주요 엔드포인트

Method	Endpoint	설명
GET	/api/graphs	그래프 목록 조회
GET	/api/graphs/:id	특정 그래프 조회
POST	/api/graphs	새 그래프 저장
PUT	/api/graphs/:id	그래프 수정
DELETE	/api/graphs/:id	그래프 삭제

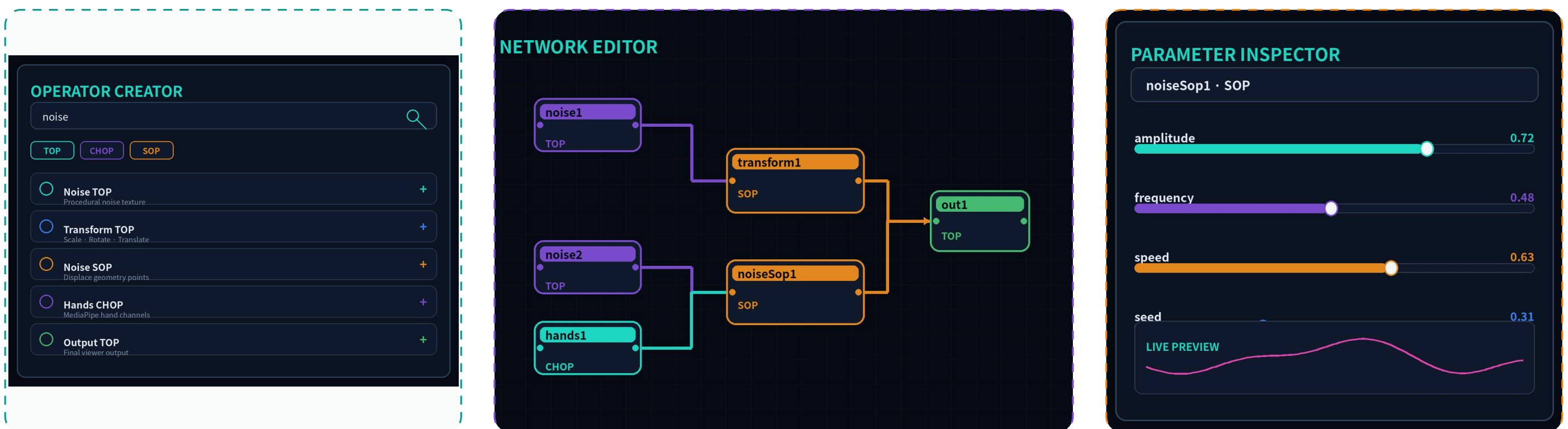
보안

- 1 요청
- 2 JWT 검증
- 3 User ID

2. ReactFlow 기반 노드 편집기

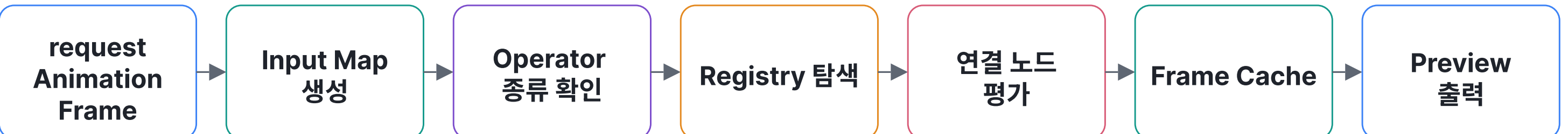


노드 생성·연결·선택·삭제와 Inspector 연동이 가능한 편집기를 구현했습니다.

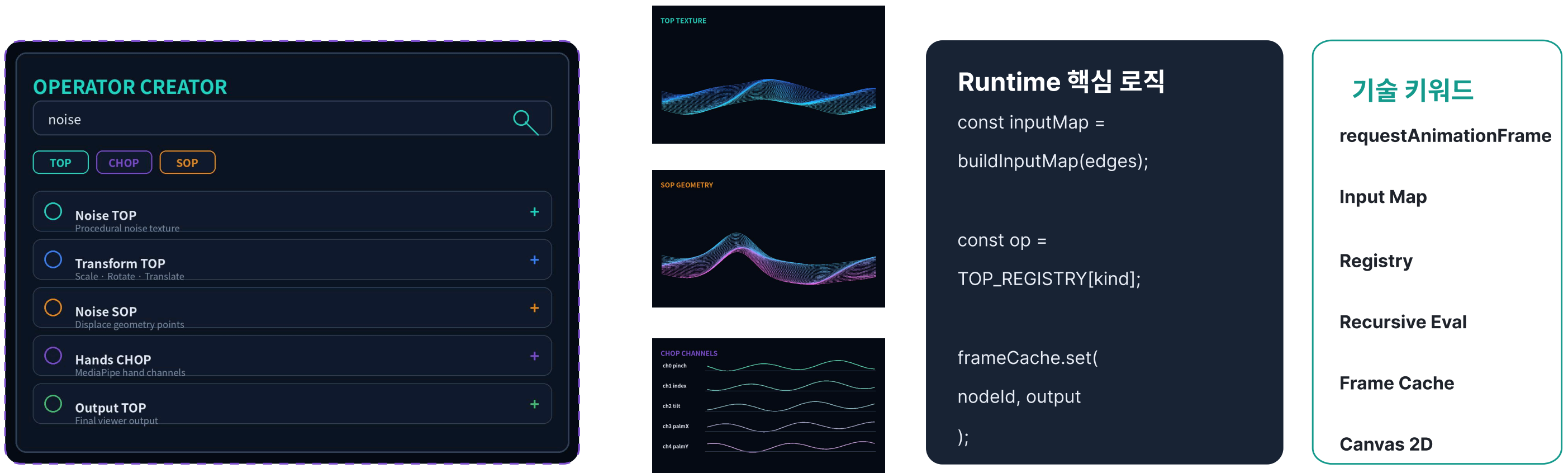


ReactFlow · useNodesState · useEdgesState · addEdge · Zustand · Snapshot History

3. 실시간 Runtime 평가

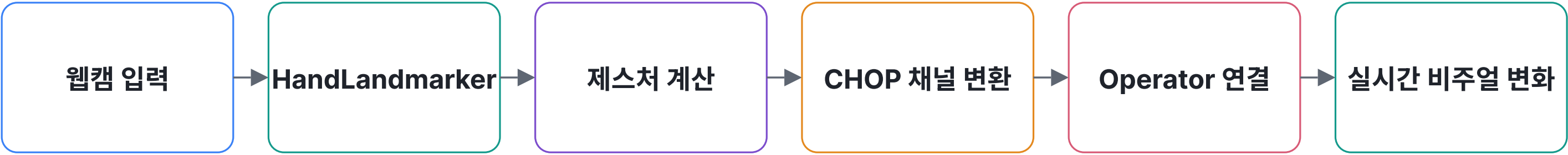


프레임마다 연결 관계를 평가하고, 노드별 미리보기와 최종 Viewer를 갱신했습니다.



노드 연결 정보를 실제 데이터 흐름으로 변환하는 실행 모델로 구현했습니다.

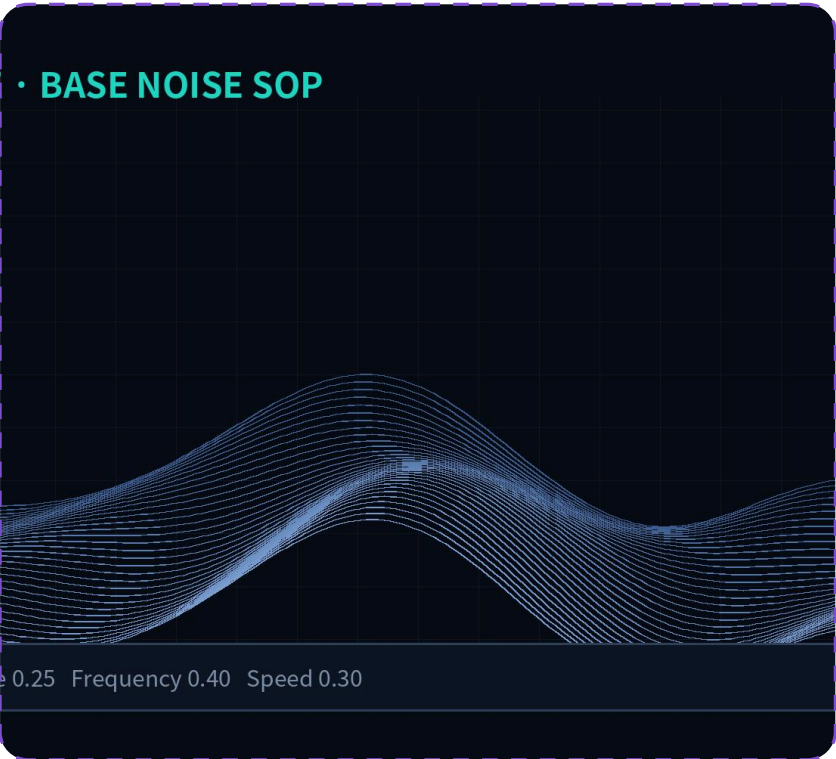
4. 손동작 데이터 처리 흐름



웹캠에서 추출한 손 랜드마크를 거리·높이·각도 값으로 계산한 뒤
0~1 범위의 CHOP 채널로 변환해 여러 Operator 파라미터에 연결했습니다.

Hands CHOP · 13 channels · reusable input system

5. 제스처 채널과 파라미터 매핑



주요 매핑 예시

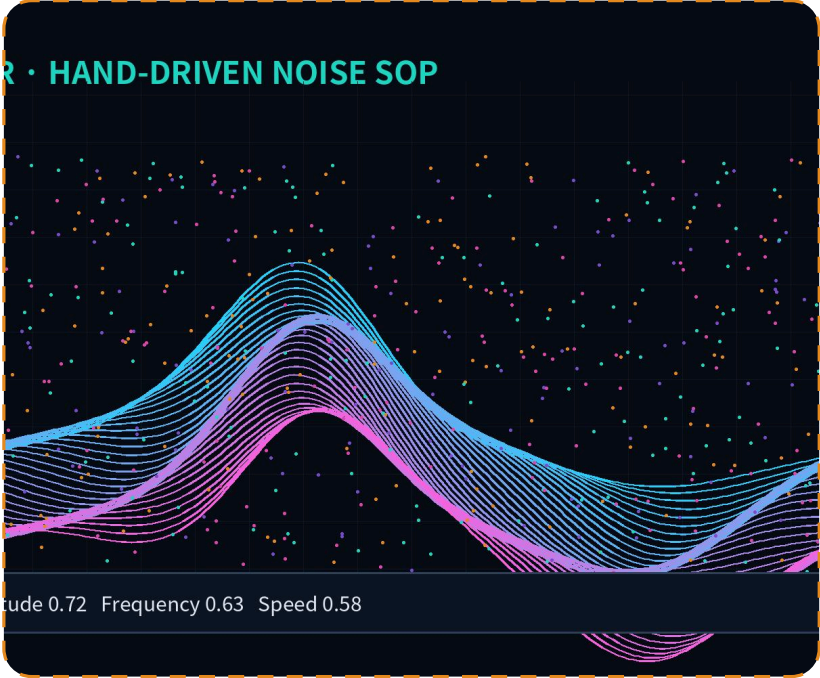
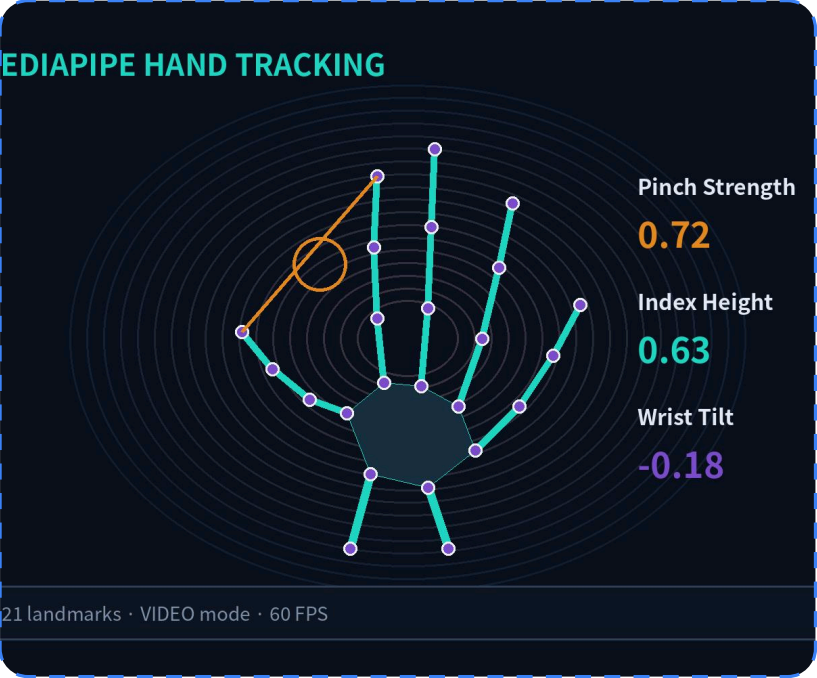
CHOP 채널	손동작	제어 대상
ch0	엄지·검지 Pinch	Noise Amplitude
ch1	검지 높이	Noise Frequency
ch2	손목 기울기	Animation Speed

Landmark → 거리·각도 계산 → 정규화 → CHOP Channel → Params

핵심 코드

```
detectForVideo()  
  
pinchDistance  
  
0~1 normalize  
  
channels[0]  
  
applyChopDrive
```

6. 손 제스처 인터랙션 결과



입력 추상화

웹캠 로직을 Hands CHOP으로 분리

데이터 채널화

손동작을 0~1 채널 값으로 변환

재사용 가능한 연결

TOP·SOP 파라미터에 반복 활용

손동작을 UI 이벤트가 아닌 CHOP 데이터로 추상화해 재사용 가능한 입력 시스템으로 구현했습니다.